COGR 2008: Practical Answers

Modules 1-5 and 7-9

Week 1: Unix essentials

Tutorial 1: directories and paths

ls lists visible entries; ls -a includes dotfiles. mkdir -p unixstuff/backups creates both directories. cd . stays in the current directory, cd .. moves to its parent, and pwd displays the absolute working path. ~ expands to the current user's home directory, so ls ~ lists that home and ls ~/.. lists its parent. A relative path is interpreted from the current directory; an absolute path starts at /. These operations were exercised inside a temporary directory to keep the original materials intact.

Reference: Tutorial One (https://www.cs.sfu.ca/~ggbaker/reference/unix/unix1.html).

Tutorial 2: files and searches

```
cp science.txt science.bak
mv science.bak backups/
mkdir tempstuff
rmdir tempstuff
head –n 5 science.txt
tail –n 15 science.txt
grep –in science science.txt
grep –ivc science science.txt
wc –w science.txt
wc –l science.txt
```

cp duplicates content; mv relocates or renames a file. rm removes a file and rmdir removes an empty directory. A nonempty backups directory cannot be removed with rmdir. head -n 5 changes the default ten lines to five; tail -n 15 prints the last fifteen. grep -i ignores case, -n adds line numbers, -v inverts the match, and -c counts matching lines. A matching-line count is different from a count of all occurrences of a word. In less, /science searches and n advances to the next match.

Reference: Tutorial Two (https://www.cs.sfu.ca/~ggbaker/reference/unix/unix2.html).

Tutorial 3: redirection and pipelines

```
printf '%s\n' pear apple banana > list1
printf '%s\n' plum apple >> list1
sort < list1 > sorted_list
sort list1 | uniq > unique_list
```

> replaces the destination, >> appends, and < supplies a file as standard input. A pipe connects one command's standard output to the next command's standard input. Sorting before uniq makes equal lines adjacent so repeated entries can be removed. To combine two existing lists and retain distinct entries: cat list1 list2 | sort | uniq > biglist.

Tutorial 4: wildcards and help

ls *.txt selects matching non-hidden names in the current directory. ? matches one character, so science.?xt matches science.txt. The shell performs this expansion before executing ls. Quoting a pattern, as in find . -name '*.txt', passes the pattern to find instead. man ls opens the command's manual; man -k keyword searches manual descriptions. File names are case-sensitive on typical Linux filesystems, while the usual macOS volume is case-insensitive.

Tutorial 5: permissions and jobs

chmod 600 science.txt grants only its owner read/write access. chmod 700 backups grants only its owner directory read/write/search access. On directories, read lists names, search/execute permits traversal, and write together with search permits creating or removing entries, subject to restrictions such as the sticky bit.

Appending & starts a background job. Ctrl-Z suspends a foreground job, bg resumes it in the background, jobs lists shell jobs, and fg brings one forward. kill -TERM PID requests termination. kill -KILL PID forces termination without allowing cleanup. The script exercised stop, continue, terminate, and wait on its own child.

Reference: Tutorial Five (https://www.cs.sfu.ca/~ggbaker/reference/unix/unix5.html).

Tutorial 6: other utilities

df -h . reports space on the containing filesystem; du -sk . reports directory usage. file science.txt identifies the file type. diff science.txt backups/science.bak checks for content changes. gzip -c science.txt > science.txt.gz creates a compressed copy, and gzip -dc science.txt.gz | cmp - science.txt checks the decompressed content. find . -type f -name '*.txt' searches recursively. Quota and print utilities depend on the host's services.

The public tutorial index is Unix Tutorial for Beginners (https://www.cs.sfu.ca/~ggbaker/reference/unix/). It covers the six topic groups above.

Week 2: build automation

Q1. Display commands without executing them

Use make -n, also called make --dry-run. For a particular target, use make -n run. Make prints the recipes it would normally execute. It does not print recipes for targets that are already up to date. Recursive make and recipes explicitly marked for execution have special dry-run behaviour; the Java recipes here contain neither.

The local dry run printed java DriverATM without running the program.

Q2. Explain javac -g SharedAccount.java

javac compiles Java source into JVM bytecode, normally writing SharedAccount.class. -g includes debugging information for line numbers, source files, and local variables so a debugger can relate the bytecode to the source. It does not start the program. Run bytecode using java DriverATM for this exercise.

Reference: Oracle javac documentation (https://docs.oracle.com/en/java/javase/25/docs/specs/man/javac.html).

Q3. Implement make run even if a file named run exists

run must be a phony target and must depend on the build:

```
.PHONY: all run clean

run: all
    $(JAVA) DriverATM
```

The complete Makefile is week02/Makefile. It also makes clean phony and records dependencies between the Java classes. Recipes begin with a tab. run: all ensures compilation finishes before execution. .PHONY prevents an unrelated file called run or clean from suppressing the recipe.

Build and timestamp exercises

The three supplied Java files were copied without changing their program logic. make clean all compiled them, and make run executed both transactions. The recorded run deposited 100, increasing the balance from 1000 to 1100, then withdrew 100, returning it to 1000. Thread order can vary, but synchronization makes each transaction indivisible with respect to the other transaction.

The phony-target exercise created files named run and clean. Execution still ran and cleaning still removed the class files. The temporary files were removed afterwards.

With the original sample Makefile, touching SharedAccountClient.java rebuilds that class alone. This completed Makefile also declares that DriverATM.class depends on it, so a client change rebuilds the client and driver. Touching a file changes its modification time, not its content; make uses those timestamps to decide whether a target is stale.

Week 3: Git version control

Q1. Count Paul Smith's commits in GNU Make

```
git clone https://github.com/mirror/make.git
cd make
git log --format='%an' | grep -x 'Paul Smith' | wc -l
```

The available full-history local clone produces 1,208 matching commits reachable from its current HEAD, at revision c63a5bc6a2881d515bb3020ed477fcba08fb2f3d. That revision's author date is 2024-08-04T23:48:51-04:00. It is an existing snapshot, not a claim about today's upstream total. git rev-parse --is-shallow-repository returned false. The pipeline emits one author name per commit, matches the complete name, and counts those lines. This counts commits, including matching merge commits, rather than changed files or lines.

Reproduce the count with bash week03/count_commits.sh /path/to/make.

Q2. What does git commit -a replace?

It stages modifications and deletions of already tracked files and then creates a commit. It combines the relevant tracked-file staging normally done with git add -u and a subsequent git commit. It does not add untracked files: a newly created file must first be staged with git add filename. git add records content in the index; git commit records the staged snapshot in repository history. Without -m, Git normally opens an editor for the message.

Reference: Git commit documentation (https://git-scm.com/docs/git-commit).

Q3. Compare the three log commands

Command	Meaning
git log	All commits reachable from HEAD, with the usual default ordering.
git log HEAD~4..HEAD	Commits reachable from HEAD excluding everything reachable from its fourth first-parent ancestor.
git log HEAD..HEAD~4	Commits reachable from that ancestor but not from HEAD; empty because the ancestor is already reachable from HEAD.

In a linear history the middle command shows the latest four commits. In a merged history it can show more than four: the local exercise shows six because commits from the merged side branch are also reachable. ~4 follows the first parent four times; a two-dot range is a reachability difference, not a fixed commit count.

Local Git exercises

The Git exercise covered initialization, staging, committing, cached diffs, an experimental branch, divergent edits, a merge conflict and its resolution, a tag, a local clone, and a pull. It also used a local bare repository to practice pushes and merges.

The actual exercise history is preserved in week03/git-exercise.bundle, including both branches. To inspect a separate clone:

```
git clone week03/git-exercise.bundle /tmp/cogr-git-review
git -C /tmp/cogr-git-review log --graph --oneline --all
```

Hosted GitLab collaboration

Week 4: testing and bug tracking

Q1. Explain each supplied assertion

In test_count_students, response.status_code == 200 checks that a request to / succeeds. int(response.data) <= n_max converts the response body to an integer and checks that it does not exceed the total capacity: 40 + 20 + 20 + 40 = 120. A nonnumeric response also fails through a conversion error.

In test_count_teachers, the status assertion checks that /teacher succeeds. The body assertion checks the entire response, including wording and instructor count. The supplied expected count of three is inconsistent with the app's one lecturer plus one tutor. It was corrected to b'There are 2 instructors', as the handout instructs fixing the test rather than changing the app.

Q2. Include the minimum class size

Replace the upper-bound assertion with:

```
assert 0 <= int(response.data) <= n_max
```

This also rejects negative totals. The application's current data is hard-coded at 120, so this assertion checks its lower and upper bounds. Both current tests passed locally.

Q3. Makefile implementation

```
PYTHON ?= python3
.PHONY: all test deploy clean

all: test
    @trap '$(MAKE) clean' EXIT; $(MAKE) deploy

test:
    @$(PYTHON) -m pytest -v > unit_test.txt 2>&1; result=$$?; cat unit_test.txt; exit $$result

deploy:
    $(PYTHON) -m flask --app prac run --host 127.0.0.1

clean:
    rm -f unit_test.txt
```

make test captures standard output and standard error in unit_test.txt, displays the saved result, and preserves pytest's exit code so failing tests fail the build. $$ passes a literal dollar sign through make to the shell. make deploy starts Flask locally and runs until stopped. make clean removes the temporary log. make all requires passing tests before deployment and installs an exit trap to clean the log after the deployment command exits. Using this ordered recipe avoids starting the server before tests finish under parallel make.

The application was checked using Flask's test client. deploy is a local development server, not a hosted deployment.

Bug-tracker comparison

OpenProj: its SourceForge bug tracker (https://sourceforge.net/p/openproj/bugs/) provides open/closed searches and columns for milestone, status, owner, dates, and priority. The fetched open list includes ticket 580 and ticket 233, illustrating that a ticket's age and its open status are separate facts.

OpenProj closed-ticket example:

GIMP: GNOME/gimp on GitHub (https://github.com/GNOME/gimp) is a read-only mirror. The branch listing showed master, gimp-3-2, and topic branches such as bruno/mac-win-accent. Their names suggest main development, release maintenance, and focused changes respectively; these purposes are an inference from names rather than verified policies. GIMP's official bug page (https://www.gimp.org/bugs/) directs bug reports to GNOME GitLab and distinguishes open issues, closed issues, platform filters, and feature proposals.

GIMP activity statistics:

Blender:

Week 5: Jupyter and debugging

Q1. Why does Jupyter show / when started from ~/prac05?

The file browser's / is the root of the directory tree exposed by the Jupyter server, not necessarily the operating system's filesystem root. With the ordinary configuration, launching the server from ~/prac05 makes that directory the server's root. Consequently the notebook browser displays it as /, even though its absolute OS path is under the user's home directory. An explicitly configured server root can override this default.

Reference: Jupyter: running the notebook (https://docs.jupyter.org/en/latest/running.html).

Q2. Explain the backtrace and recursion

The Variables panel shows the values in the selected stack frame, including n and the lists referenced by source, target, and auxiliary. The call stack/backtrace shows the active nested function calls. Repeated move frames with decreasing n demonstrate recursion. In the first move of the three-disk run, the active frames have n=3, then n=2, then n=1; they also assign different source/target roles to the same peg lists. The n=0 base case returns without moving a disk.

The recursive algorithm moves the top n-1 disks aside, moves the largest remaining disk to the destination, and moves the n-1 disks onto it. Its recurrence is T(n) = 2T(n-1) + 1, giving 2^n - 1 moves, so three disks require seven moves.

week05/Hanoi.ipynb contains the three executable practical cells with saved outputs. The completed run ended with A and B empty and C equal to [3, 2, 1]. The notebook contains the recursive function and its seven moves.

To perform the interactive part, open the notebook in JupyterLab, enable the debugger, place a breakpoint at target.append(source.pop()) in the function cell, and run the cells in order. Defining the function does not execute its body, so the breakpoint is reached only when the third cell calls it.

Q3. Combine VM forwarding and the SSH tunnel

```
ssh -p 2522 -N -L 127.0.0.1:8080:localhost:8888 tonystark@localhost
```

VirtualBox must forward host TCP port 2522 to the VM's SSH port 22. -p 2522 connects to that forwarded SSH service; -N starts no remote command. -L listens on the host's loopback port 8080 and forwards traffic through SSH to localhost:8888 as seen from the VM. Start Jupyter inside the VM on port 8888, keep the tunnel open, and use http://localhost:8080 with Jupyter's authentication token. The host and guest occurrences of localhost identify different machines in their respective contexts.

The question sheet specifies guest port 8888. The last example in the practical handout instead says 8080; use the actual guest Jupyter port, which is normally 8888.

Week 7: processes, scheduling, and synchronization

Practical Q1. Explain the CPU-consuming loop

seq 1 100000 produces 100,000 loop values. Each iteration executes expr 1 + 1, which evaluates to 2, and redirects its output to /dev/null. The arithmetic result is discarded. Most of the cost comes from repeatedly launching the external expr program, including process creation and scheduling, rather than performing one addition. Five background copies create runnable work that competes for CPU time. The provided week07/prac07.sh uses the requested loop, with an optional iteration-count argument.

Practical Q2. Nice level and CPU utilization

A larger nice value generally gives a normal Linux process a lower scheduling weight. Under CPU contention it normally receives a smaller share than an otherwise comparable process with a lower nice value. Nice values are relative scheduling preferences, not fixed CPU percentages. A lower-priority process can still use an idle CPU fully; multiple cores and waiting for child processes can also affect the observation. Increasing a process's priority usually needs additional permission.

nice -n 10 command starts at an adjusted nice level. On Linux, renice -n 10 -p PID requests a nice value of 10; exact renice option semantics should be checked on other Unix platforms. The Raspberry Pi Linux run succeeded: five ordinary workers started at nice 0, a sixth at nice 10, and two workers were changed to nice 10 and 15. The ps snapshots in week07/processes.txt show lower measured CPU percentages for the adjusted workers during that short run; the exact percentages depend on contention and sampling.

Practical Q3. Kill all the practical's workers

week07/kill_workers.sh uses pgrep restricted to the current user, then checks each matched process's command line against the absolute path to this practical's prac07.sh. It sends SIGTERM only to matching workers. Checking the path avoids killing unrelated programs whose names happen to contain the same text. The bracketed pattern [p]rac07[.]sh prevents matching the literal search pattern itself. SIGKILL is suitable for the handout's forced-termination demonstration but should not replace graceful termination when cleanup matters.

week07/run_process_demo.sh starts five normal workers and one nice-adjusted worker, records process data, renices two workers, forcibly ends one, and terminates the others. It cleans up its own children even if a step fails. The Raspberry Pi run completed with all six workers exited; its transcript is week07/processes.txt. tegrastats remains Jetson-specific.

In-class Q1. Round robin with a 500 ms quantum

Assume the CPU is busy whenever a job is runnable and a change between jobs costs 10 ms. Execution efficiency means useful CPU work divided by elapsed CPU-plus-switch time; it is not completed-jobs throughput for jobs that never finish.

Two compute-bound jobs: a full recurring cycle is 500 + 10 + 500 + 10 = 1020 ms. Efficiency is 1000/1020 = 98.04%, and each job gets 500/1020 = 49.02% of elapsed time. The jobs are treated equally. If both arrive at time zero and job 1 starts immediately, their first-response delays are 0 and 510 ms, giving 255 ms average first response. After a job exhausts a slice, the time until its next slice is 10 + 500 + 10 = 520 ms. These are different response-time definitions.

A compute-bound job and an editor: an editor request arriving just as the compute job starts can wait 510 ms before executing, then use 20 ms. The input-arrival distribution is not specified, so a unique average interactive response cannot be calculated. If requests arrive uniformly during the compute job's 500 ms

burst, the conditional mean delay to first execution is 250 + 10 = 260 ms. An immediately ready editor can have a much shorter delay.

For an illustrative recurring cycle in which the editor becomes ready in time for its next turn, useful work is 500 + 20 = 520 ms and switching is 20 ms, giving efficiency 520/540 = 96.30%. This is conditional on the editor's unspecified input timing. One 10 ms switch is 50% of a 20 ms editor burst, but that is not a 50% overhead for the whole workload. Equal turns do not guarantee acceptable interactive latency. A smaller quantum improves responsiveness at the cost of more switching; an MLFQ or an interactive-priority policy can respond to short bursts while preserving longer slices for CPU-bound work. Periodic priority boosts help prevent starvation.

In-class Q2. Three round-robin tasks

All jobs arrive at time zero. The quantum is 19 and each switch costs 1. The following chart follows the supplied solution's convention of charging a switch after every CPU segment, including the final exit:

Task	CPU interval	Switch interval	Remaining CPU
1	0-19	19-20	47
2	20-39	39-40	5
3	40-59	59-60	24
1	60-79	79-80	28
2	80-85	85-86	0
3	86-105	105-106	5
1	106-125	125-126	9
3	126-131	131-132	0
1	132-141	141-142	0

Useful execution is 66 + 24 + 43 = 133 units. Under that convention there are nine switches and 142 elapsed units, so switching consumes 9/142 × 100 = 6.34%. Taking completion at the ends of the charged slots gives (86 + 132 + 142)/3 = 120 units.

If completion means the instant a task finishes its CPU work, completion times are 85, 131, and 141, with mean 119 units. If no final switch is charged after the last task, the makespan is 141 with eight intervening switches, giving 8/141 × 100 = 5.67% switching overhead. These alternatives explain the one-unit convention difference rather than changing the CPU schedule.

In-class Q3. First ten MLFQ iterations

An iteration here means one dispatch, matching the supplied solution. Queue front is shown on the left. A CPU-bound process using its entire quantum moves down a level; C returns to Q1 after its one-unit I/O. I/O overlaps CPU execution. As in the supplied table, an I/O completion does not preempt a lower-level process in the middle of its current quantum. A strictly preemptive MLFQ would produce a different timeline, so this assumption is necessary. Context-switch overhead is unspecified and treated as zero.

The parenthesized letter identifies the process that ran during that row, corresponding to the requested circled process. Queues and I/O are shown at the end of the dispatch, with remaining CPU amounts.

Iteration	Time	Ran	Q1: 2 units	Q2: 4 units	Q3: 8 units	In I/O
0	0	-	A:10, B:20, C:10	-	-	-
1	0-2	(A)	B:20, C:10	A:8	-	-
2	2-4	(B)	C:10	A:8, B:18	-	-
3	4-5	(C)	-	A:8, B:18	-	C:9 until 6
4	5-9	(A)	C:9	B:18	A:4	-
5	9-10	(C)	-	B:18	A:4	C:8 until 11
6	10-14	(B)	C:8	-	A:4, B:14	-
7	14-15	(C)	-	-	A:4, B:14	C:7 until 16
8	15-19	(A)	C:7	-	B:14	-
9	19-20	(C)	-	-	B:14	C:6 until 21
10	20-28	(B)	C:6	-	B:6	-

A has completed by iteration 8. C still needs six CPU bursts and six I/O units after iteration 10. week07/scheduling.py reproduces the timeline and queue snapshots.

In-class Q4. Semaphore outcomes

Treat each displayed instruction as atomic. down(s) blocks when the semaphore is zero; up(s) increments it and permits a waiting operation to proceed. Initial state: x=0, y=23, z=0, m=1, ml=0, mx=1. week07/semaphore_outcomes.py enumerates the complete instructions, including semaphore operations.

(a) Final x is 24 or 25. Both processes use m around their updates, so the two critical sections cannot overlap. P1 then P2 gives z=x; z++; x=z followed by y++; x=y, ending at 24. P2 then P1 first sets x=24, then increments it to 25. Both completed orders release m and neither deadlocks.

(b) Final x is 25 only. P1 initially blocks on down(ml). P2 executes down(m); y++; x=y; up(ml), establishing x=24 before P1 proceeds. P1 then executes down(ml); z=x; z++; x=z; up(m), establishing x=25. This is an ordering constraint between processes.

(c) Final x is 1, 24, or 25. The up operations at the beginning and down operations at the end do not protect the updates. Example full schedules are:

```
x=1:
P1 up(ml), z=x, z++;
P2 up(ml), y++, x=y;
P1 x=z, down(m);
P2 down(ml).

x=24:
P1 up(ml), z=x, z++, x=z, down(m);
P2 up(ml), y++, x=y, down(ml).

x=25:
P1 up(ml);
P2 up(ml), y++, x=y;
P1 z=x, z++, x=z, down(m);
P2 down(ml).
```

(d) Successful completion gives x=24 or x=25, and deadlock is also reachable. If P1 acquires both locks first, its full critical section precedes P2 and x ends at 24. If P2 acquires both first, the final value is 25. A deadlock schedule is P1 down(mx), P2 down(m), then P1 blocks on down(m) and P2 blocks on down(mx). Here x remains zero, but this is a blocked intermediate state rather than a completed result. Requiring both processes to acquire locks in the same order prevents this circular wait.

In-class Q5. Deadlock and starvation

Deadlock is a state in which a set of activities cannot progress because each waits for a resource or event that another member of that set must release or produce. Starvation means one activity waits indefinitely while other work can continue, for example because a priority policy never selects it. Fair scheduling can address some forms of starvation but cannot resolve a circular lock dependency on its own.

Deadlock requires mutual exclusion, holding resources while waiting for more, lack of preemption, and circular wait. Prevention breaks at least one condition: impose a global lock order, acquire needed resources together, avoid holding locks across waits, or use appropriate try-lock/retry designs. Avoidance can instead admit allocations only when the state remains safe, provided resource requirements are known.

Recovery can abort a selected process or transaction, roll it back, or preempt a recoverable resource. Recovery must restore application invariants; forcibly unlocking arbitrary live program state is unsafe. A single thread can deadlock by attempting to acquire the same nonrecursive mutex twice before releasing it, or by waiting for an event that only it can generate.

Week 8: filesystems and disk storage

Practical Q1. Find the SD-card mount and filesystem type

The practical uses mount | grep mmcblk to select SD-card partition mounts. In a line such as /dev/mmcblk0p1 on / type ext4 (...), / is the mount point and ext4 is the filesystem type. sudo fdisk -l lists partitions but does not itself establish their current mount points. cat /etc/fstab shows configured mounts, which should be compared with the live mount output.

On the Raspberry Pi used for the follow-up run, findmnt -no SOURCE,FSTYPE / reported /dev/mmcblk0p2 ext4; mount independently showed that partition mounted at / as ext4. The boot partition /dev/mmcblk0p1 was vfat. These are Pi observations in week08/mounts.txt, not measurements of the course Jetson.

Practical Q2. Explain the image-counting loop

find . -type d produces the directory names below the current directory. The shell substitutes that output into the for loop; each iteration prints a directory and then runs a recursive search for names ending in .jpg, ignoring case. wc -l counts the output lines. Because each directory search is recursive, a parent's count includes images in all its descendants. The totals therefore overlap and must not be added to obtain a unique overall image count.

The original for i in $(find ...) and unquoted $i break on directory names containing whitespace or wildcard characters. Without -type f, directories whose names end in .jpg also count. week08/count_images.sh corrects these problems using NUL-delimited paths, quoted arguments, and -type f. A local fixture containing two JPEG files, one inside a directory with spaces, produced a root count of two and a subdirectory count of one.

Practical Q3. Why avoid nesting tmux?

An inner and outer session normally share the Ctrl-b prefix. The outer session intercepts it, making it less obvious which session receives commands; nested panes and status bars also become confusing. Use another window or pane in the existing session. If nesting is intentional, send a prefix through to the inner session or configure distinct prefixes. tmux a || tmux attaches to an existing session or creates one if attachment fails.

week08/tmux_demo.sh uses a separate tmux socket to avoid touching personal sessions. The Raspberry Pi run succeeded: its counter advanced from 3 to 5 while another window was selected and no client was attached. The two captures and window list are in week08/tmux.txt.

Object detection exercise

In-class Q1. Fairness and request ordering

For a continuous random stream, a conventional fairness ranking is FCFS, C-SCAN, SCAN, SSTF, from most to least fair. FCFS preserves arrival order. C-SCAN tends to give more uniform waits across positions. SCAN services requests in both directions, giving position-dependent waits. SSTF can indefinitely postpone distant requests under a stream of nearby arrivals. Fairness and minimum head movement are different objectives.

With the given finite queue, initial head position 9, upward motion, and reversal/wrap at the last requested cylinder rather than the physical disk edge:

Algorithm	Service order	Total movement
FCFS	11, 15, 18, 6, 13, 7, 5, 8	39 cylinders
SSTF	8, 7, 6, 5, 11, 13, 15, 18	17 cylinders
SCAN	11, 13, 15, 18, 8, 7, 6, 5	22 cylinders
C-SCAN	11, 13, 15, 18, 5, 6, 7, 8	25 cylinders

The specified endpoint behaviour is often called LOOK and C-LOOK. The C-SCAN movement includes the return from 18 to 5, although no requests are serviced during that return. The script week08/disk_calculations.py computes these orders and distances.

In-class Q2. Small queues and locality

With exactly one pending request there is no scheduling choice, so all algorithms choose that request. With two or three requests the algorithms need not become FCFS. For example, from cylinder 9, if requests arrive in order 18 then 8 and both are pending, FCFS chooses 18 while SSTF chooses 8. Small queues simply reduce opportunities for reordering.

Spatial locality usually reduces the practical differences because requests cluster within a small region, making long seeks less common. The exact effect still depends on head position and the arrival pattern; a persistent nearby stream can make SSTF's starvation of a distant request especially visible.

In-class Q3. Disk timings and transfer rate

6000 rpm / 60 = 100 revolutions per second, giving 10 ms per revolution. A random request waits on average half a revolution, so average rotational delay is 5 ms. Combined seek plus rotation is 10 ms, so average seek time is 10 - 5 = 5 ms.

The average complete block read takes 6 seconds / 400 = 15 ms. Subtracting seek and rotation leaves 5 ms transfer time. At 10,000 bytes per block, raw transfer rate is 10,000 / 0.005 = 2,000,000 bytes/s = 2 MB/s. If the question uses 10 KiB instead of decimal 10 kB, the equivalent result is 2,048,000 bytes/s; the time calculations do not change.

In-class Q4. Inode maximum file size and block 1200

A 4096-byte pointer block holds 4096/4 = 1024 pointers. The inode can therefore address:

```
10 direct blocks
+ 1024 singly indirect blocks
+ 1024^2 doubly indirect blocks
+ 1024^3 triply indirect blocks
= 1,074,791,434 data blocks
```

Maximum file content size is 1,074,791,434 × 4096 = 4,402,345,713,664 bytes, approximately 4.402 TB or 4.00391 TiB. Pointer blocks consume additional disk space but are not part of the file's data length. The stated disk capacity in the preceding independent question does not constrain this question's different hypothetical inode structure.

Direct and single-indirect addressing cover the first 10 + 1024 = 1034 blocks. The one-based 1200th data block is therefore in the double-indirect region. With only the inode cached, read the double-indirect root block, its first leaf pointer block, and then the requested data block: three disk reads. Its leaf entry is the 166th data pointer in that leaf, or zero-based index 165. Cached pointer blocks could reduce the reads.

In-class Q5. A crash during journal recovery

A committed transaction remains in the journal until its home-location updates are safely persisted and the journal space can be reclaimed. After another crash, recovery can replay the still-valid committed record again. Redo operations must be idempotent, such as writing the recorded block contents, so repeating them does not corrupt an already updated block.

An incomplete transaction without a valid commit record is not replayed. If a recovery crash occurs after some home blocks have been written, retained committed records allow the rest to be recovered on the next boot. Safe checkpointing and journal reuse must distinguish complete records from partial writes, and ordering/flush guarantees must ensure that data does not appear committed before the required log records are durable. Thus repeated crashes can be tolerated by a correctly implemented protocol; deleting needed log data too early breaks that guarantee.

Week 9: memory management

Practical Q1. Explain each NumPy allocation

np.zeros(123456789) uses the default float64, with eight bytes per element. Each array's data buffer therefore occupies 987,654,312 bytes, about 0.920 GiB, excluding array-object and allocator overhead. The arithmetic operations create new arrays rather than changing my_array.

Command	Live arrays	Total live array data
import numpy as np	None from the exercise	0 bytes, plus imported-library overhead
my_array = np.zeros(123456789)	Original	987,654,312 bytes
my_array1 = my_array + 1	Original, +1	1,975,308,624 bytes
my_array2 = my_array + 2	Original, +1, +2	2,962,962,936 bytes
my_array3 = my_array + 3	Original, +1, +2, +3	3,950,617,248 bytes
del my_array3	Original, +1, +2	2,962,962,936 bytes
del my_array1	Original, +2	1,975,308,624 bytes

The table gives logical array-buffer sizes, not measured RSS or guaranteed swap use. Demand-zero pages, allocator caching, and OS accounting mean physical usage need not immediately equal the allocated size. Producing the nonzero arrays touches their output pages. On a 2 GB Jetson the working set can exceed available RAM, leading to swapping or allocation failure if insufficient backing storage exists. del removes a reference; a buffer becomes reclaimable when no references remain, but it need not immediately reduce the process's RSS.

The Raspberry Pi completed the handout's 123,456,789-element run. With all four arrays live, logical array data was 3,950,617,248 bytes and process RSS was about 2.99 GB. The initially allocated zero array added almost nothing to RSS until the later operations touched its pages. After deleting my_array3 and my_array1, RSS fell to about 1.01 GB. Pi swap usage stayed at zero throughout; the separate free inspection showed 7.6 GiB of RAM and 2.0 GiB of zram swap. The measurements are in week09/memory_full.json and week09/memory_inspection.txt. These are Pi results; the 2 GB Jetson could behave differently.

Reference for the default array type: NumPy zeros (https://numpy.org/doc/stable/reference/generated/numpy.zeros.html).

Practical Q2. Why do the parent and child branches both print?

os.fork() creates a child process. Both processes resume after the call, but the return value differs: the parent receives the child's positive PID and the child receives zero. Therefore the parent executes the if n > 0 branch and the child executes else. One process does not execute both branches. Their outputs can appear in either order because they are independently scheduled. The completed demonstration also calls waitpid so the parent reaps the child before exiting.

Reference: Python os.fork (https://docs.python.org/3/library/os.html#os.fork).

Practical Q3. Same address, different values; integers versus floats

Each process has a separate virtual address space after fork. The same numerical virtual address in two processes can map to different physical storage. Initially many pages can be shared with copy-on-write; writes create private copies as needed. Consequently equal printed addresses across processes do not

establish that the processes share a mutable Python object.

Python integers and floats are immutable. x += 1 or x += 2 rebinds x to the result rather than changing the original numeric object's value. In CPython, small integers are cached, so the preexisting objects for 11 and 12 have distinct identities. Floats produced after a fork can be allocated at corresponding virtual locations by the two copied allocators, producing equal numerical addresses for 11.0 and 12.0 in different processes. This outcome is implementation- and run-dependent; it is not guaranteed for every float operation.

The Mac run matched the handout's pattern: parent and child integer result IDs differed, while their float result IDs were equal. In the Raspberry Pi Linux run, the float result IDs differed too, confirming that equality is not guaranteed. Both runs produced the expected independent parent and child values. Pi values and addresses are in week09/fork_output.txt. Within the initial single process, x=10; y=10 shared the same cached integer object, then y += 1 changed only y's binding. The ctypes demonstration retrieved 10 from the still-live x object.

id() is generally an object identity; its interpretation as an address is a CPython implementation detail. References: Python id (https://docs.python.org/3/library/functions.html#id) and CPython integer objects (https://docs.python.org/3/c-api/long.html). Do not assume a specific small-integer cache boundary across interpreter versions.

Homework Q1. Virtual and physical addresses

A virtual address is an address interpreted within a process's address space. A physical address identifies a location in physical memory. The MMU translates a virtual page number through the current address-space mapping and combines the selected physical frame with the page offset. Page tables can map the same virtual page number to different frames for different processes. Thus two variables can have identical numerical virtual addresses yet independent stored values. Processes can also deliberately share a physical frame through explicit shared-memory mappings.

Homework Q2. Swap-backed virtual memory and thrashing

Swap allows less-active memory pages to be moved out of RAM so more programs, or a larger total allocation, can remain supported than would fit entirely in physical memory. Demand paging can keep the actively used working set in RAM while leaving less-used data on storage. It does not make storage as fast as RAM, and not all virtual pages necessarily need swap: some are file-backed or can be recreated.

Page faults introduce storage latency, page-management work, and sometimes dirty-page writes. A swap file consumes disk space; on flash, write traffic also affects wear. If half of all memory references require swap retrieval, performance is extremely poor because storage access is orders of magnitude slower than a RAM access. A simplified model is effective access time = (1-p) × RAM-access-time + p × page-fault-service-time. For illustration, at p=0.5, 100 ns RAM and 1 ms fault service give about 0.50005 ms per reference, roughly 5000 times the RAM-only access time. Actual performance depends on hardware, locality, caching, and fault overlap; these numbers are illustrative, not measured.

At a zero swap-retrieval rate, execution is not delayed by reads from swap, although ordinary cache misses, file-backed page faults, and other overhead remain possible. Thrashing occurs when active working sets exceed available frames and repeated faults evict pages that are soon needed again. Reduce the active workload or its working set, add RAM, use locality-aware algorithms, or adjust admission/working-set policies. A larger swap file alone may avoid allocation failure without curing thrashing.

Homework Q3. Memory and I/O protection

The OS installs page-table mappings with access permissions such as user/supervisor, readable, writable, and executable. On each access, the MMU checks translation and permissions; an invalid or forbidden access traps into the kernel. Different processes normally receive different page-table contexts, with

translation caches handled appropriately during context switches.

User mode cannot normally alter the protected page tables, enter privileged CPU modes directly, or execute privileged I/O instructions. Device registers mapped into memory are kept outside ordinary user mappings unless the kernel explicitly grants access. Applications request I/O through system calls, and kernel drivers validate and perform it. An IOMMU can additionally restrict device DMA; this is distinct from a CPU MMU's protection of process accesses.

Homework Q4. Implement second-chance FIFO

Keep resident pages in a circular list of frames and maintain a clock hand. Each frame has a reference bit that the MMU sets on an access. A hit marks that page referenced. On a page fault, inspect the frame at the hand: if its bit is one, clear it and advance; if its bit is zero, choose it as the victim. Install the new page and advance the hand. A free frame can be used without eviction. Write a dirty victim to backing storage before reusing its frame, and update page tables and invalidate stale translations as required.

Second chance approximates recency without moving pages on every reference or recording exact access timestamps. It needs O(F) frame entries and reference bits for F frames. A replacement may scan O(F) frames; if every reference bit is set, a complete pass clears them and the next visit selects a victim. Page lookup is normally assisted by the hardware mapping rather than a linear search. Sampling or clearing accessed bits can require architecture-specific page-table and TLB handling. The algorithm does not distinguish one recent use from many uses, may evict useful pages under some patterns, and is not exact LRU.

week09/second_chance.py provides a small runnable model with three frames. For accesses 1, 2, 3, 1, 4, 2, 5, the trace has five page faults. Its list lookup is deliberately simple and costs O(F); a production implementation would use a direct mapping for resident-page lookup.